Control de iluminación con el sensor de luz del smartphone

Asignatura: Desarrollo de Aplicaciones Móviles
Evaluación: Segundo corte
Proyecto: Integración de sensores de smartphones, Firebase Realtime Database y ESP32

1. Título del proyecto e integrantes

Título: Sistema inteligente de iluminación: control automático de una lámpara mediante el sensor de luz ambiental de un smartphone, Firebase Realtime Database y ESP32. Presentado y realizado por Juan Angel Ortiz Vargas, código estudiantil 20211196244.

Docente: Johan Julian Molina Mosquera

Repositorio del proyecto: https://github.com/Ju4ngel/LuzInteligente.git

2. Descripción y funcionamiento del proyecto

Propósito y problema que resuelve

El sistema automatiza el encendido y apagado de una lámpara según la luz del ambiente. En viviendas, oficinas y espacios domóticos es común dejar luces encendidas sin necesidad o no tenerlas cuando oscurece. Este proyecto utiliza el sensor de luminosidad que ya trae un smartphone para medir la luz del lugar y decidir de forma automática si la lámpara debe estar encendida.

Sensor y variables capturadas

- **Sensor utilizado:** sensor de luz ambiental del smartphone Android (`Sensor.TYPE_LIGHT`), que entrega la iluminancia en **lux**.
- **Variables en Firebase** (nodo `/iluminacion`):

Variable	Tipo	Descripción
`lux`	número	Luminosidad medida por el smartphone
`lampara`	booleano	Estado de la lámpara calculado por el ESP32
`umbral_encendido`	número	Por debajo de este valor se enciende (50 lux)
`umbral_apagado`	número	Por encima de este valor se apaga (150 lux)
`timestamp`	número	Marca de tiempo de la última lectura (servidor)

Procesamiento de los datos

1. La app Android lee el sensor, muestra el valor en pantalla y lo envía a Firebase solo si pasaron al menos 500 ms y el valor cambió al menos 1 lux, para no saturar la base de datos.
2. El ESP32 consulta `/iluminacion` cada segundo y aplica una lógica de **histéresis** con dos umbrales:

- Si `lux < 50` → lámpara **ENCENDIDA**.
- Si `lux > 150` → lámpara **APAGADA**.
- Entre 50 y 150 lux → se **mantiene el estado anterior**, lo que evita parpadeos cuando la luz está cerca de un único umbral.

3. El ESP32 actualiza `lampara` en Firebase, activa o desactiva el actuador y muestra el estado en la pantalla OLED.

Respuesta de los actuadores

- **Actuador principal:** LED blanco conectado a un pin digital del ESP32, que cumple el papel de la lámpara.
- **Indicador:** pantalla OLED que muestra, por ejemplo, `Luminosidad: 38 lux` y `Lampara: ENCENDIDA`.

> ***Nota sobre el actuador:*** *por costo y disponibilidad de materiales se utilizó un LED blanco como actuador en lugar de un módulo relé con lámpara de baja tensión. La lógica de control es idéntica: para una lámpara real bastaría agregar un módulo relé controlado por el mismo pin digital.*

3. Modelo de conexión cliente-servidor



Figura 1. Diagrama de Flujo del Proyecto.

Intercambio de información

- **Cliente 1 (Android → Firebase):** se autentica de forma anónima y escribe `lux` y `timestamp` en `/iluminacion` mediante `updateChildren`, de modo que no sobrescribe los umbrales ni el estado de la lámpara.
- **Servidor (Firebase Realtime Database):** almacena el valor más reciente y aplica reglas de seguridad: solo usuarios autenticados pueden leer y escribir.
- **Cliente 2 (ESP32 ↔ Firebase):** se autentica con un usuario de correo y contraseña, lee `/iluminacion`, decide el estado de la lámpara y escribe `lampara`.

Reglas de seguridad configuradas:

```
{
  "rules": {
    "iluminacion": {
      ".read": "auth != null",
      ".write": "auth != null"
    }
  }
}
```

4. Diseño del circuito electrónico ESP32-actuadores

Componentes: ESP32 DevKit (ESP-WROOM-32, USB tipo C), pantalla OLED I2C de 1,30" (controlador SH1106), LED blanco, resistencia de [100 a 220] Ω, protoboard y cables Dupont.

Componente	Terminal	Pin del ESP32
OLED	VCC	3V3
OLED	GND	GND
OLED	SCK (SCL)	GPIO22
OLED	SDA	GPIO21
LED	Ánodo (pata larga)	GPIO23, a través de la resistencia
LED	Cátodo (pata corta)	GND

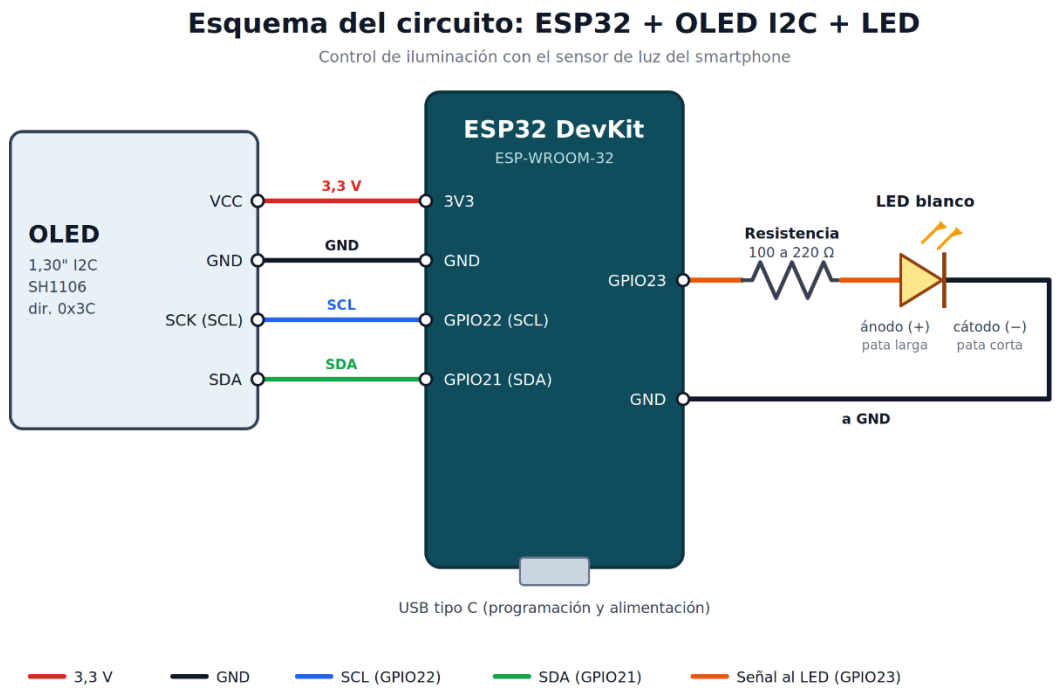


Figura 2. Esquema Circuital del Proyecto

5. Explicación del circuito electrónico

- **ESP32 DevKit:** microcontrolador con Wi-Fi integrado. Se conecta a internet, consulta Firebase y controla los periféricos. Se alimenta y programa por USB tipo C.
- **Pantalla OLED I2C (SH1106, 128x64):** muestra la luminosidad y el estado de la lámpara. Usa el bus I2C: SDA en GPIO21 y SCL en GPIO22, con dirección `0x3C`. Se alimenta con 3,3 V.
- **LED blanco:** representa la lámpara. Se enciende cuando el pin GPIO23 está en nivel alto.
- **Resistencia limitadora:** limita la corriente del LED para proteger tanto al LED como al pin del ESP32. Va en serie entre GPIO23 y el ánodo.
- **Protoboard y cables:** permiten armar el circuito sin soldar.

Lógica de activación: el ESP32 lee `lux` y los umbrales desde Firebase. Si `lux` es menor que el umbral de encendido, pone GPIO23 en alto (LED encendido). Si `lux` es mayor que el umbral de apagado, lo pone en bajo (LED apagado). Entre ambos umbrales conserva el estado anterior.

6. Diagramas de flujo o pseudocódigos

Proceso 1. Smartphone → Firebase

```
INICIO
  Obtener SensorManager y el sensor de luz (TYPE_LIGHT)
  SI no existe sensor ENTONCES mostrar aviso
  Iniciar sesión anónima en Firebase
  Registrar el listener del sensor
  REPETIR cada vez que el sensor cambia:
      lux = valor del sensor
      Mostrar lux en pantalla
      SI autenticado Y pasaron >= 500 ms Y |lux - ultimoLux| >= 1 ENTONCES
          Enviar a /iluminacion: lux y timestamp
          Guardar ultimoEnvio y ultimoLux
      FIN SI
  FIN REPETIR
FIN
```

Proceso 2. Firebase → ESP32

```
INICIO
  Configurar pines (LED como salida) e iniciar la OLED
  Conectar a la red Wi-Fi
  Iniciar sesión en Firebase (correo y contraseña)
  REPETIR cada 1 segundo:
      SI Firebase listo ENTONCES
          Leer /iluminacion: lux, umbral_encendido, umbral_apagado
          nuevo = estado actual de la lámpara
          SI lux < umbral_encendido ENTONCES nuevo = ENCENDIDA
          SINO SI lux > umbral_apagado ENTONCES nuevo = APAGADA
          SI nuevo cambió ENTONCES
              Activar o desactivar el LED
              Escribir lampara en Firebase
          FIN SI
          Mostrar lux y estado en la OLED
      FIN SI
```

```
  FIN REPETIR
FIN
```

7. Codificación del sistema

7.1 Android Studio (Kotlin): `MainActivity.kt`

```
package com.juanangel.luzinteligente

import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.database.FirebaseDatabase
import com.google.firebase.database.ServerValue
import kotlin.math.abs

class MainActivity : AppCompatActivity(), SensorEventListener {

    private val dbUrl = "https://luz-inteligente-2f973-default-rtdb.firebaseio.com"

    private lateinit var sensorManager: SensorManager
    private var lightSensor: Sensor? = null
    private lateinit var tvLux: TextView
    private lateinit var tvEstado: TextView

    // Inicialización perezosa: evita usar Firebase antes de que esté listo
    private val auth by lazy { FirebaseAuth.getInstance() }
    private val ref by lazy {
        FirebaseDatabase.getInstance(dbUrl).getReference("iluminacion")
    }

    private var autenticado = false
    private var ultimoEnvio = 0L
    private var ultimoLux = -1f

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        tvLux = findViewById(R.id.tvLux)
        tvEstado = findViewById(R.id.tvEstado)

        // Obtiene el sensor de luz ambiental del teléfono
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        lightSensor = sensorManager.getDefaultSensor(Sensor.TYPE_LIGHT)
        if (lightSensor == null) {
            tvEstado.text = "Este dispositivo no tiene sensor de luz"
        }
        iniciarSesion()
```

```
    }

    // Autenticación anónima requerida por las reglas de la base de datos
    private fun iniciarSesion() {
        if (auth.currentUser != null) {
            autenticado = true
            tvEstado.text = "Conectado a Firebase"
            return
        }
        auth.signInAnonymously()
            .addOnSuccessListener {
                autenticado = true
                tvEstado.text = "Conectado a Firebase"
            }
            .addOnFailureListener { e ->
                tvEstado.text = "Error de login: ${e.message}"
            }
    }

    override fun onResume() {
        super.onResume()
        lightSensor?.let {
            sensorManager.registerListener(this, it,
SensorManager.SENSOR_DELAY_NORMAL)
        }
    }

    override fun onPause() {
        super.onPause()
        sensorManager.unregisterListener(this)
    }

    // Se ejecuta cada vez que el sensor entrega una lectura
    override fun onSensorChanged(event: SensorEvent) {
        val lux = event.values[0]
        tvLux.text = "${lux.toInt()} lux"

        // Limita la frecuencia de envío: cada 500 ms y solo si cambia >= 1 lux
        val ahora = System.currentTimeMillis()
        val pasoTiempo = ahora - ultimoEnvio >= 500
        val cambioSignificativo = abs(lux - ultimoLux) >= 1f

        if (autenticado && pasoTiempo && cambioSignificativo) {
            ultimoEnvio = ahora
            ultimoLux = lux
            // updateChildren solo modifica estos campos, sin borrar umbrales ni
estado
            val datos = mapOf(
                "lux" to lux.toInt(),
                "timestamp" to ServerValue.TIMESTAMP
            )
            ref.updateChildren(datos)
                .addOnFailureListener { e ->
                    tvEstado.text = "Error al enviar: ${e.message}"
                }
```

```
        }
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
}
```

Funciones principales: `onCreate` prepara la interfaz y el sensor; `iniciarSesion` autentica de forma anónima; `onResume` y `onPause` registran y liberan el sensor; `onSensorChanged` muestra el valor y lo envía a Firebase con control de frecuencia.

Interfaz: `activity_main.xml` contiene un `LinearLayout` con dos `TextView`: `tvLux` (valor en lux) y `tvEstado` (estado de la conexión). Se requiere el permiso `INTERNET` en `AndroidManifest.xml`.

7.2 Arduino IDE (ESP32): `luz_esp32.ino`

Las credenciales están en un archivo `secrets.h` que no se publica en el repositorio.

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>      // Pantalla 1.30" con controlador SH1106
#include <Firebase_ESP_Client.h>
#include "addons/TokenHelper.h"
#include "secrets.h"              // WIFI_SSID, WIFI_PASSWORD, API_KEY, DATABASE_URL,
USER_EMAIL, USER_PASSWORD

#define PIN_LED 23
#define OLED_ADDR 0x3C

Adafruit_SH1106G display = Adafruit_SH1106G(128, 64, &Wire, -1);
FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

float lux = 0;
int umbralOn = 50;      // Se enciende por debajo de este valor
int umbralOff = 150;    // Se apaga por encima de este valor
bool lampara = false;
bool primeraLectura = true;
unsigned long ultimaConsulta = 0;

// Dibuja dos líneas de texto en la OLED
void mostrar(const String &l2, const String &l3) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SH110X_WHITE);
  display.setCursor(0, 0);
  display.println("LUZ INTELIGENTE");
  display.setCursor(0, 20);
  display.println(l2);
  display.setCursor(0, 40);
  display.println(l3);
  display.display();
}
```

```
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, LOW);

  Wire.begin(21, 22);                        // SDA = GPIO21, SCL = GPIO22
  if (!display.begin(OLED_ADDR, true)) {
    Serial.println("No se encontro la OLED");
  }
  mostrar("Conectando WiFi...", "");

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi OK");

  // Configuración y autenticación de Firebase
  config.api_key = API_KEY;
  config.database_url = DATABASE_URL;
  auth.user.email = USER_EMAIL;
  auth.user.password = USER_PASSWORD;
  config.token_status_callback = tokenStatusCallback;
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);

  mostrar("Conectando a", "Firebase...");
}

void loop() {
  // Consulta una vez por segundo
  if (millis() - ultimaConsulta < 1000) return;
  ultimaConsulta = millis();
  if (!Firebase.ready()) return;

  if (Firebase.RTDB.getJSON(&fbdo, "/iluminacion")) {
    FirebaseJson &json = fbdo.jsonObject();
    FirebaseJsonData d;

    json.get(d, "lux");
    if (d.success) lux = d.floatValue;
    json.get(d, "umbral_encendido");
    if (d.success) umbralOn = d.intValue;
    json.get(d, "umbral_apagado");
    if (d.success) umbralOff = d.intValue;

    // Histéresis: entre los dos umbrales se mantiene el estado anterior
    bool nuevo = lampara;
    if (lux < umbralOn) nuevo = true;
    else if (lux > umbralOff) nuevo = false;

    if (nuevo != lampara || primeraLectura) {
      lampara = nuevo;
      primeraLectura = false;
```

```
      digitalWrite(PIN_LED, lampara ? HIGH : LOW);
      Firebase.RTDB.setBool(&fbdo, "/iluminacion/lampara", lampara);
    }

    mostrar("Luminosidad: " + String((int)lux) + " lux",
            String("Lampara: ") + (lampara ? "ENCENDIDA" : "APAGADA"));
    Serial.printf("lux=%.0f lampara=%d\n", lux, lampara);
  } else {
    Serial.println(fbdo.errorReason());
  }
}
```

Funciones principales: `setup` inicializa pines, OLED, Wi-Fi y Firebase; `loop` consulta la base de datos cada segundo y aplica la histéresis; `mostrar` dibuja el estado en la pantalla.

8. Resultados y pruebas de funcionamiento

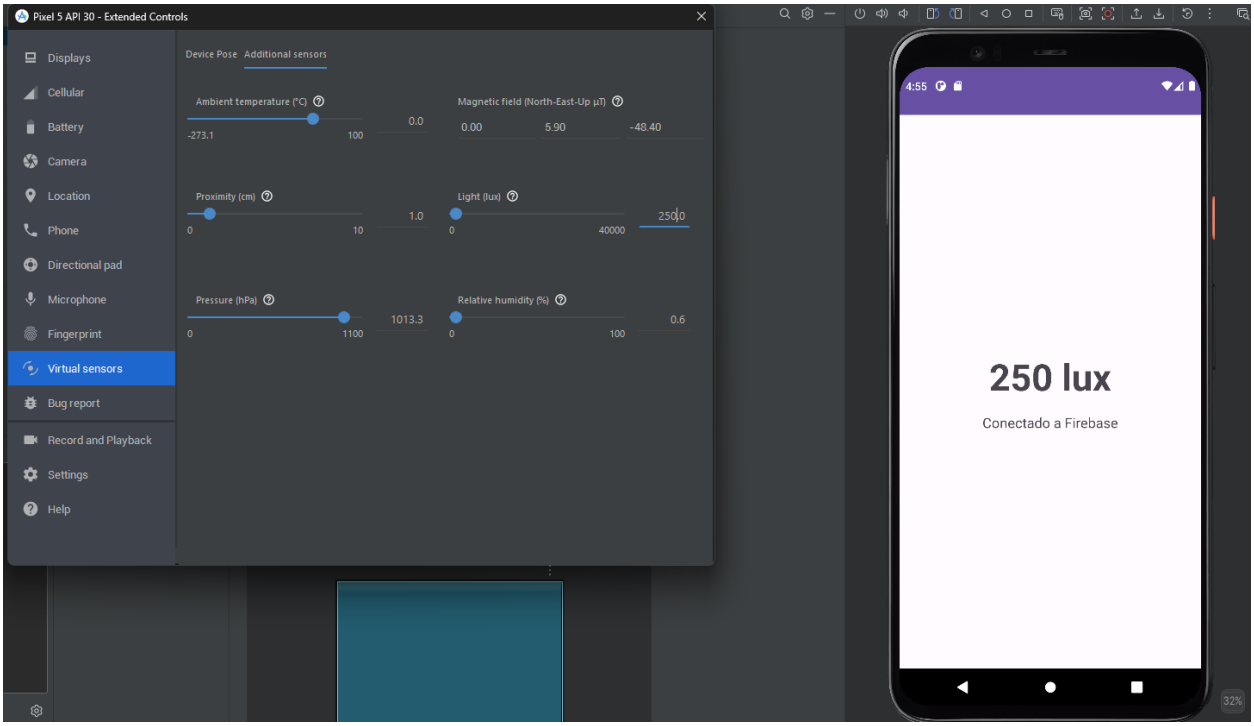
Condiciones evaluadas

Umbral de encendido: 50 lux. Umbral de apagado: 150 lux. Se simularon distintos niveles de luz y se verificó cada eslabón de la cadena: captura, transmisión, recepción y respuesta del actuador.

Prueba	Luminosidad	Estado previo	Resultado esperado	Resultado observado
1. Ambiente oscuro	20 lux	Apagada	LED encendido, OLED: ENCENDIDA	LED encendido
2. Zona intermedia (histéresis)	100 lux	Encendida	Se mantiene ENCENDIDA	LED encendido
3. Ambiente iluminado	200 lux	Encendida	LED apagado, OLED: APAGADA	LED apagado
4. Oscuridad nuevamente	30 lux	Apagada	LED encendido	LED encendido

Verificación por etapas

1. **Captura de datos:** la app muestra el valor del sensor al variar la luz.



2. **Transmisión a Firebase:** el valor `iluminacion/lux` cambia en tiempo real en la consola.



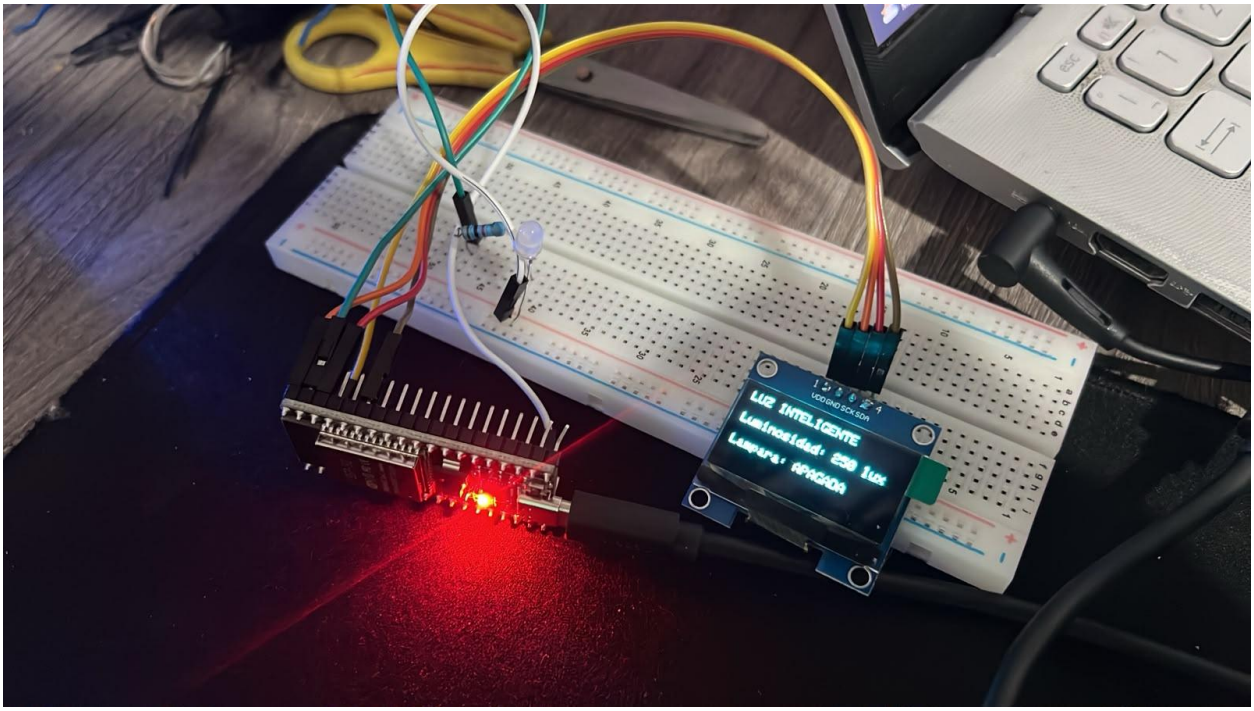
3. **Recepción por el ESP32:** el Monitor Serie muestra `lux=... lampara=...` con el valor enviado desde la app.

Salida Monitor Serie ×

Mensaje (Intro para mandar el mensaje de 'ESP32 Dev Module' a 'COM4')

```
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
lux=250 lampara=0
```

4. **Respuesta de los actuadores:** el LED y la OLED reaccionan según los umbrales.



Observaciones y conclusiones

- Latencia aproximada entre el cambio de luz y la respuesta del LED
- Problemas encontrados y cómo se resolvieron: por ejemplo, la pantalla de 1,30" usa el controlador SH1106 y requirió la librería Adafruit SH110X.
- Mejoras futuras: relé con lámpara de baja tensión, histéresis configurable desde la app, historial de datos

9. Evidencias fotográficas

- Interfaz de la apicación Android en funcionamiento

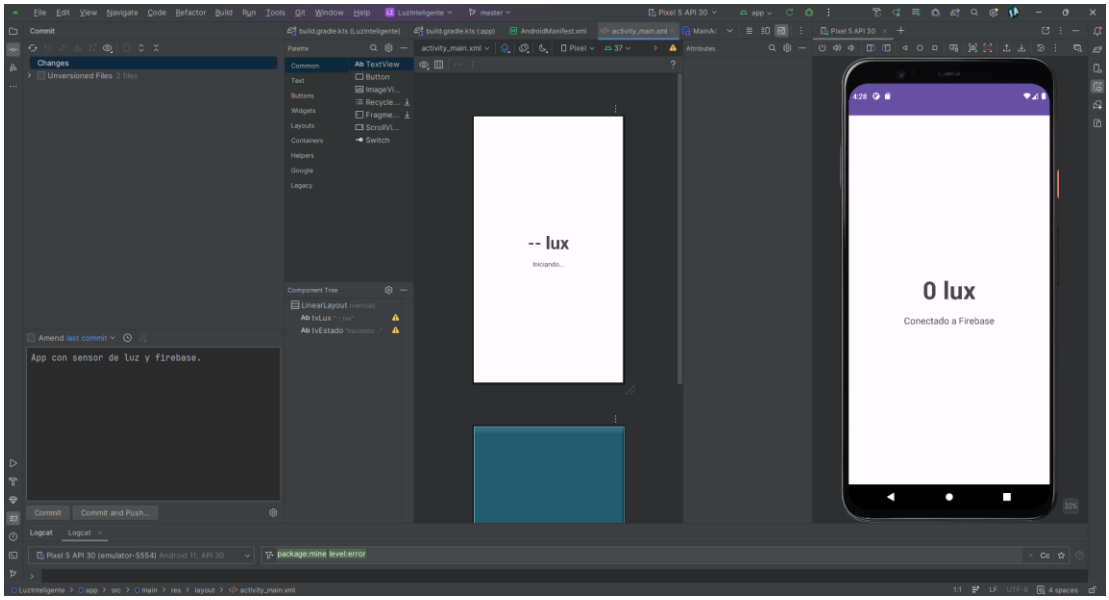


Figura 3. Interfaz y funcionamiento del aplicativo Android

- Datos registrados en Firebase Realtime Database.

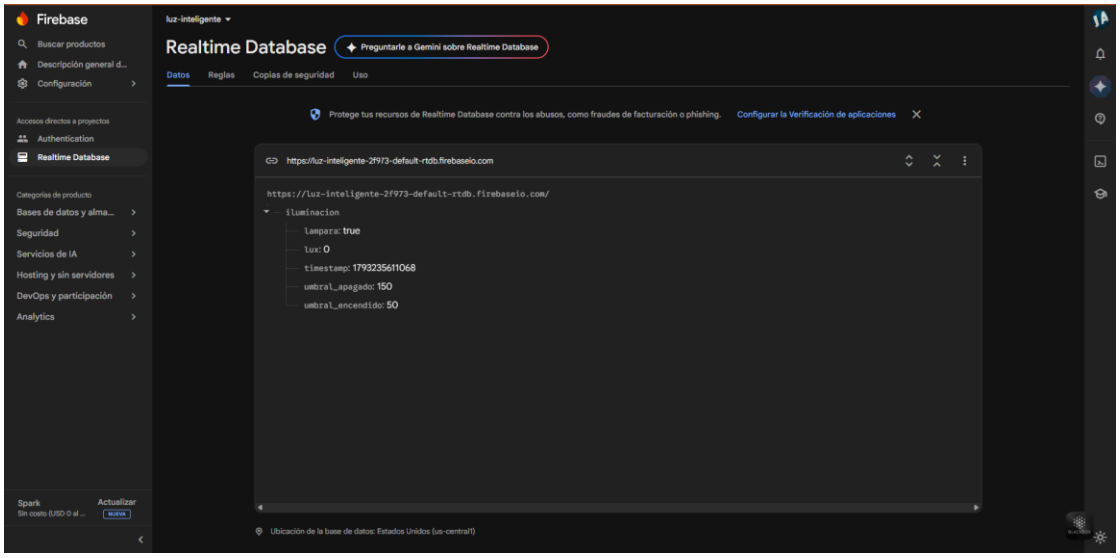


Figura 4. Base de datos funcional en 'Firebase Realtime Database'.

- Montaje físico del circuito electrónico.

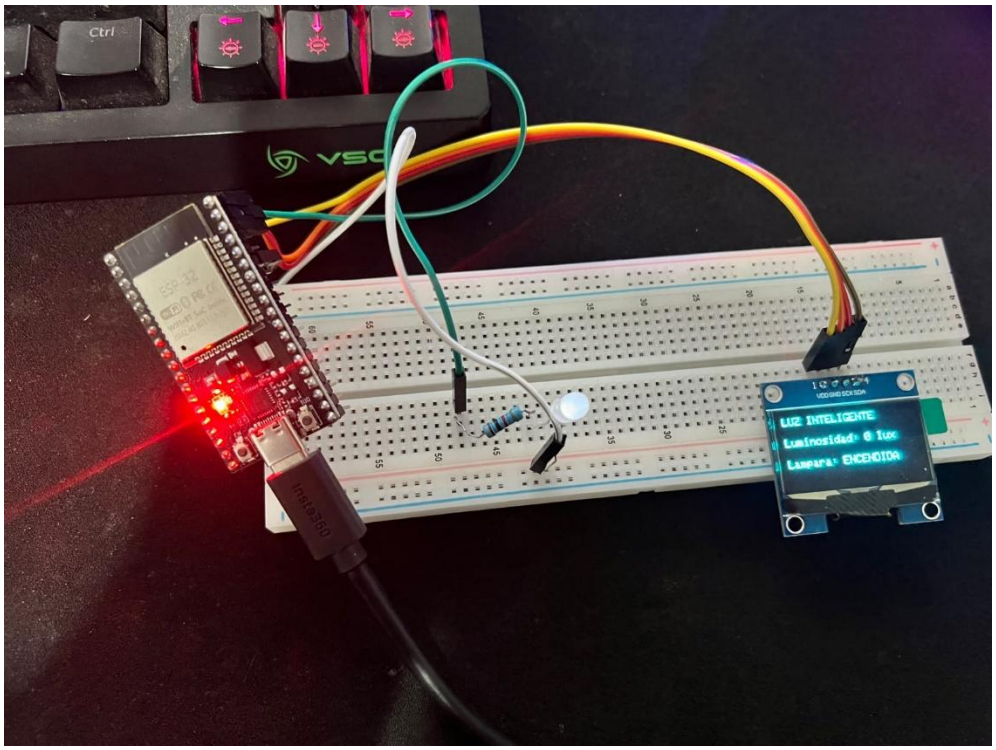


Figura 5. Montaje físico del circuito Electrónico.

- Actuadores en diferentes posiciones: LED encendido (poca luz)

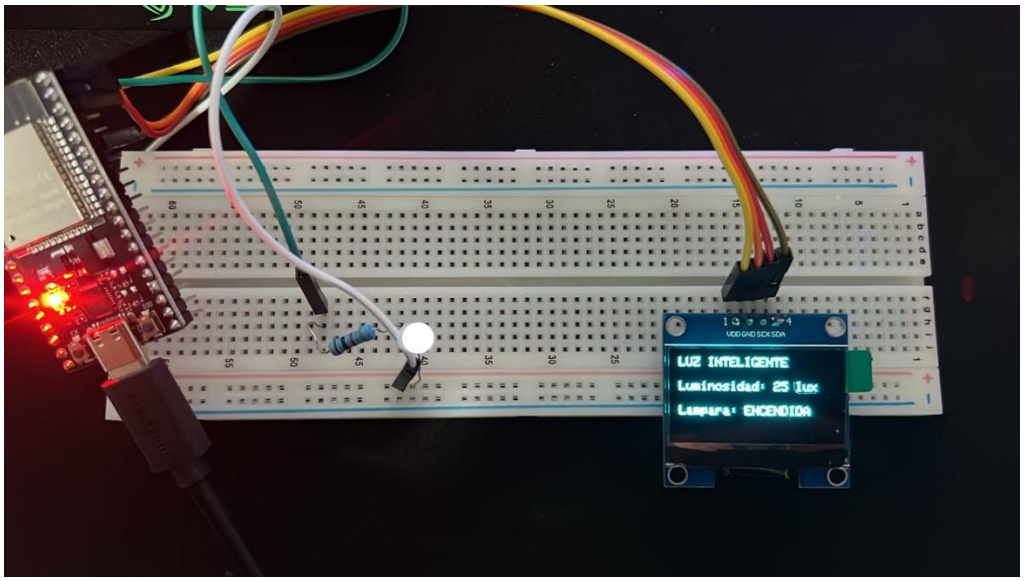


Figura 6. Encendido del LED al Detectar Poca Luminicidad.

- Actuadores en diferentes condiciones: LED apagado (mucha luz)

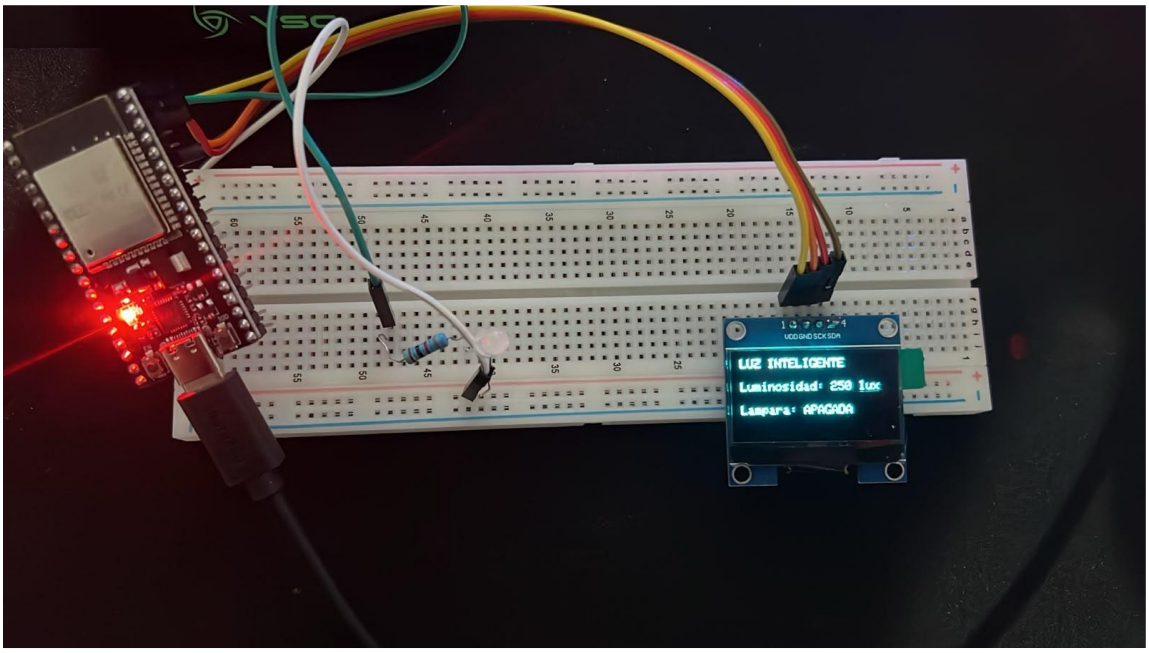


Figura 7. LED Apagado Por Censo de Alta Luminosidad

- Integración completa: celular, consola de Firebase y circuito en la misma escena

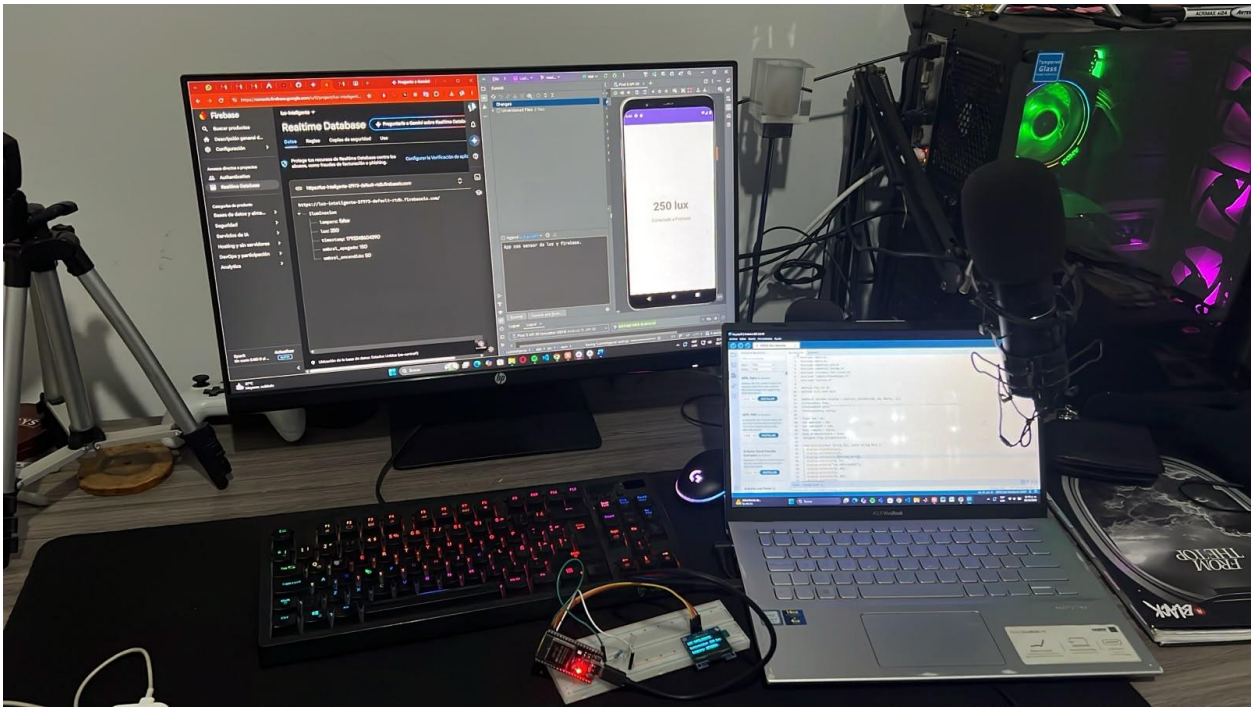


Figura 8. Esquema completo de funcionamiento físico y virtual.

Referencias

- Documentación de Firebase Realtime Database y Authentication.
- Documentación de Android: sensores de entorno (`TYPE_LIGHT`).
- Librería Firebase Arduino Client (Mobizt), Adafruit GFX y Adafruit SH110X.